

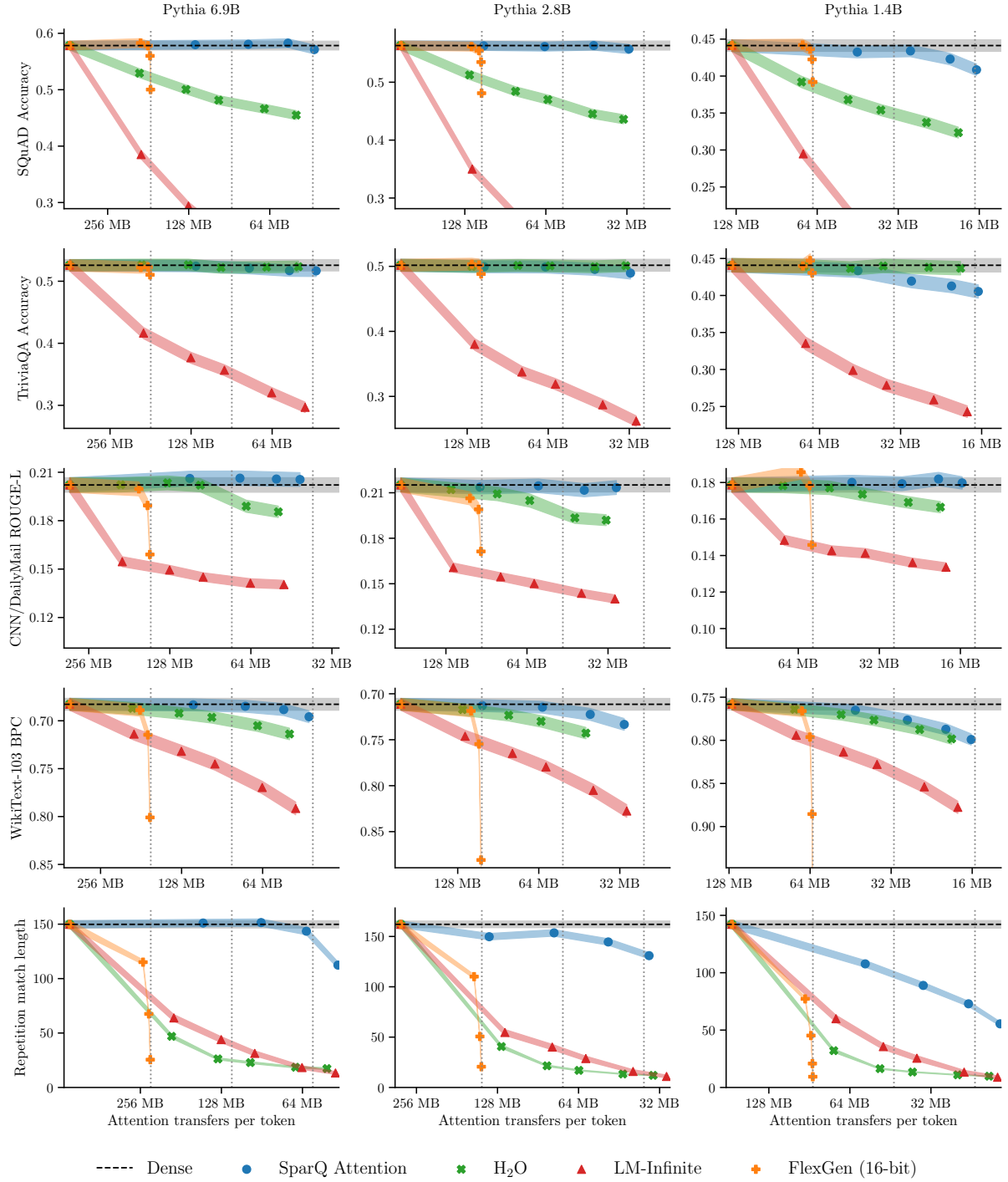


Figure A3: Compression versus performance trade-off curves over all tasks for the Pythia family of models. The y-axis minimum is set to $(0.5, 0.5, 0.5, 1.25, 0.0) \times$ the dense baseline for the tasks, reading top-to-bottom, in order to give a consistent view of the performance loss across models. Vertical dotted lines show $1/2\times$, $1/4\times$ and $1/8\times$ compression versus dense. Shaded lines show ± 1 standard error of the mean (uncertainty due to a finite test set).

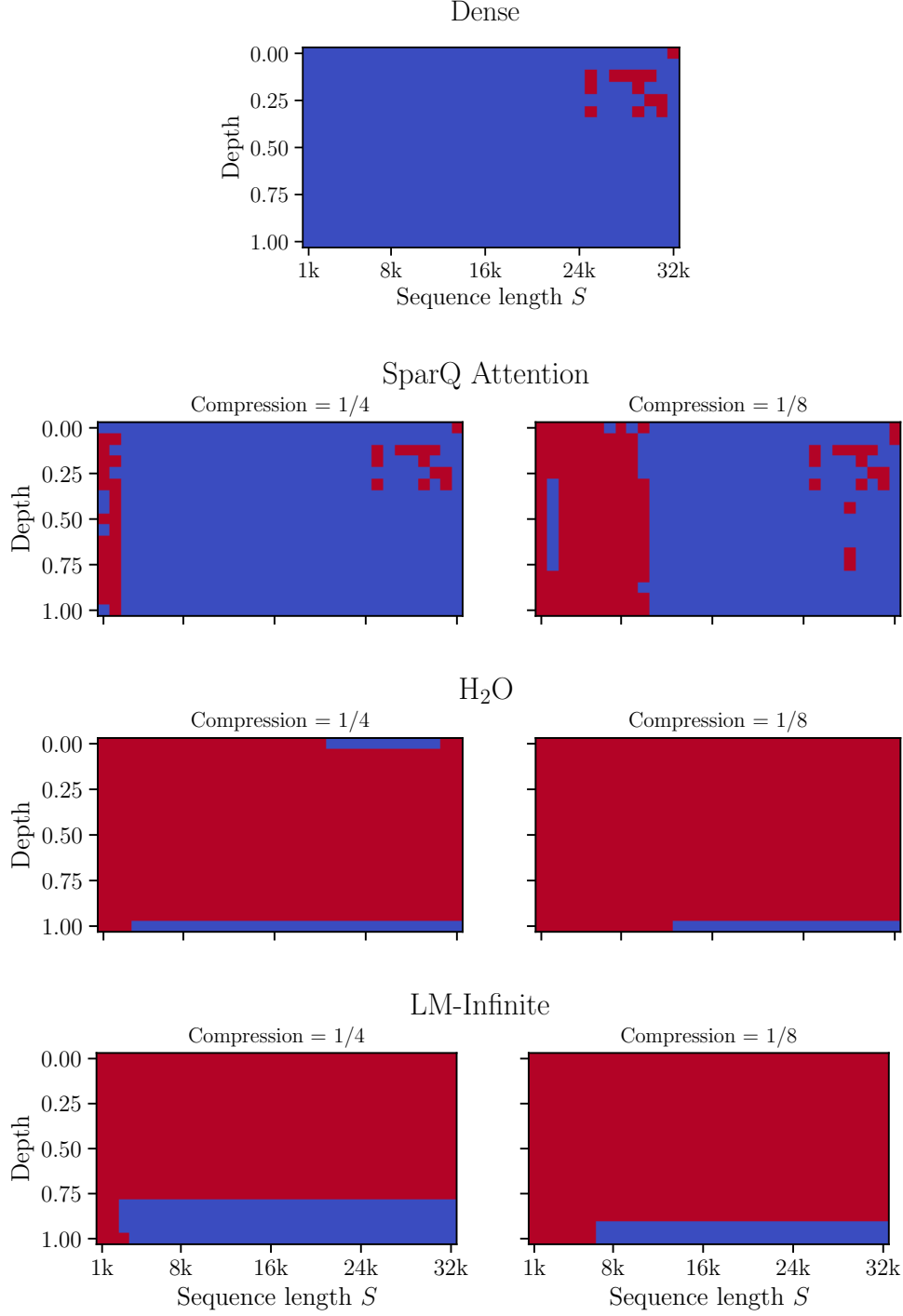


Figure A4: Heatmaps of the results for the needle in a haystack task, as outlined in [Dhinakaran \(2024\)](#). For various sequence lengths (comprising of essays by Paul Graham), the needle (which is the sequence “*The best thing to do in San Francisco is eat a sandwich and sit in Dolores Park on a sunny day*”) is inserted into the sequence at various depths, and the model is prompted to answer what the best thing to do in San Francisco is. The heatmaps show whether the model returns the correct (blue) or incorrect (red) answer. We evaluated this task over SparQ Attention, H₂O and LM-Infinite (in addition to the dense baseline) on the `togethercomputer/LLaMA-2-7B-32K` model, for two compression ratios.

B. Code

```

from torch import softmax, sqrt, tensor, topk

def gather(t, dim, i):
    dim += (dim < 0) * t.ndim
    return t.gather(dim, i.expand(*t.shape[:dim], i.shape[dim], *t.shape[dim + 1 :]))

def attn(Q, K, V, M):
    s = (Q @ K.transpose(-1, -2)) / sqrt(tensor(Q.shape[-1])) + M
    y = softmax(s, dim=-1) @ V
    return y

def sparq_attn(Q, K, V, V_mean, M, r, k):
    # Q -- (batch_size, n_kv_heads, n_heads // n_kv_heads, 1, head_size)
    # K, V -- (batch_size, n_kv_heads, 1, seq_len, head_size)

    # 1. Approximate attention scores using r largest components of Q
    i1 = topk(abs(Q).sum(dim=2, keepdim=True), r, -1).indices
    Q_hat, K_hat = gather(Q, -1, i1), gather(K, -1, i1)
    scale = sqrt(
        Q.shape[-1]
        * abs(Q_hat).sum(dim=-1, keepdim=True)
        / abs(Q).sum(dim=-1, keepdim=True)
    )
    s_hat = softmax(Q_hat @ K_hat.transpose(-1, -2) / scale + M, dim=-1)

    # 2. Gather top k positions based on approximate attention scores & run attention
    i2 = topk(s_hat.sum(dim=2, keepdim=True), k, -1).indices
    iKV = i2[:, 0, :, None]
    K, V, M = gather(K, -2, iKV), gather(V, -2, iKV), gather(M, -1, i2)
    y_ = attn(Q, K, V, M)

    # 3. Estimate the total score of the top k, and interpolate with V_mean
    alpha = gather(s_hat, -1, i2).sum(-1, keepdim=True)
    y = alpha * y_ + (1 - alpha) * V_mean
    return y

```

C. Arithmetic Intensity

Consider a full transformer layer, with N parameters, batch size B , C elements in the attention KV cache per batch element and g grouped-query heads per key-value head. This implies the arithmetic intensity:

$$\frac{\mathcal{A}}{\mathcal{M}} = \frac{BN + BCg}{N + BC} = \frac{N + Cg}{N/B + C} \quad (\text{C1})$$

We can increase arithmetic intensity by making B large, causing $\mathcal{A}/\mathcal{M}$ to approach $N/C + g$. Hence the limiting factor for large-batch transformer inference is the ratio of the KV cache size per-item to the size of the model.

We can alternatively express this in terms of the model’s basic hyperparameters. A standard transformer with model dimension d_m and sequence-length S has $N = 12(d_m)^2$ and $C = 2Sd_m/g$ (Kaplan et al., 2020). Substituting these values into Equation (C1), we get

$$\frac{\mathcal{A}}{\mathcal{M}} = \frac{6 + \rho g}{6/B + \rho} \quad (\text{C2})$$

where $\rho = S/(gd_m)$ is a variable we have introduced, and underlies the KV cache-model size relationship outlined above, determining the point at which the model becomes memory bandwidth bound. We observe that the arithmetic intensity as batch size increases approaches $g + 6/\rho$.